

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 1

«Вычисление освещенности в точках поверхности»

Выполнили студенты:

Баянов Равиль, Кузнецов Даниил

Группы № Р3434

Преподаватель: Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2025

Цель

Изучить, как вычисляется освещенность в точках точки на плоскости треугольника при освещении его точечным источником света с заданной диаграммой излучения.

Используемые формулы

1. «Цветная» интенсивность источника под углом к оси источника света:

$I(RGB, \vec{s}) = I_0(RGB) \cos \theta$, где $I_0(RGB)$ – «цветная» интенсивность источника света в направлении его оси $\vec{O}$, $\|\vec{O}\| = 1$, θ - угол между направлением распространения света и осью источника света, $\cos \theta$ – диаграмма излучения.

2. «Цветная» освещенность точки:

$E(RGB, \vec{P}_T) = \frac{I(RGB, \vec{s}) \cos \alpha}{R^2}$, где $I(RGB, \vec{s})$ – «цветная» интенсивность света, α - угол между направлением света и нормалью к освещаемой поверхности, R - расстояние от источника света до рассматриваемой точки.

3. Перевод локальных координат точки в плоскости в глобальные

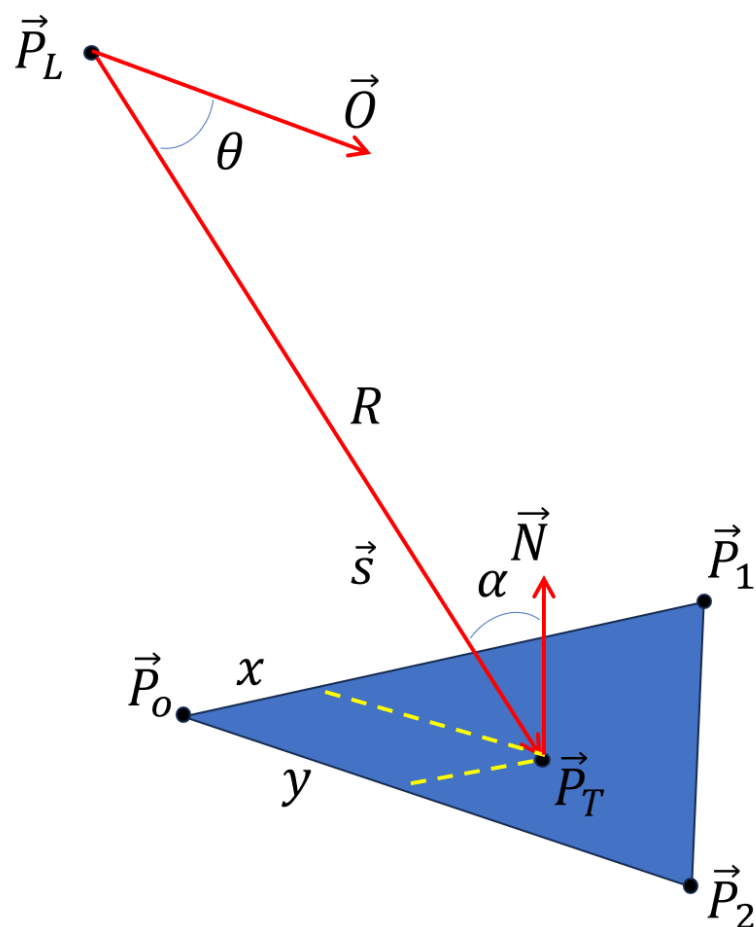
$\vec{P}_T = \vec{P}_0 + \left(\frac{\vec{P}_1 - \vec{P}_0}{\|\vec{P}_1 - \vec{P}_0\|} \cdot x + \frac{\vec{P}_2 - \vec{P}_0}{\|\vec{P}_2 - \vec{P}_0\|} \cdot y \right)$, где x и y смещения по ребрам треугольника.

4. Вычисление вектора нормали плоскости треугольника через 3 точки:

$$\vec{N} = \frac{(\vec{P}_2 - \vec{P}_0) \times (\vec{P}_1 - \vec{P}_0)}{\|(\vec{P}_1 - \vec{P}_0) \times (\vec{P}_2 - \vec{P}_0)\|}$$

5. Вектор от точки плоскости до источника света:

$$\vec{s} = \vec{P}_T - \vec{P}_L, R^2 = \|\vec{s}\|^2, \cos \alpha = \frac{\vec{s} \cdot \vec{N}}{\|\vec{s}\|}, \cos \theta = \frac{\vec{s} \cdot \vec{O}}{\|\vec{s}\|}$$



Шаги алгоритма

1. Вычисление вектора нормали

$$\vec{N} = \frac{(\vec{P}_2 - \vec{P}_0) \times (\vec{P}_1 - \vec{P}_0)}{\|(\vec{P}_1 - \vec{P}_0) \times (\vec{P}_2 - \vec{P}_0)\|}$$

2. Перевод локальных координат в глобальные

$$\vec{P}_T = \vec{P}_0 + \left(\frac{\vec{P}_1 - \vec{P}_0}{\|\vec{P}_1 - \vec{P}_0\|} \cdot x + \frac{\vec{P}_2 - \vec{P}_0}{\|\vec{P}_2 - \vec{P}_0\|} \cdot y \right)$$

3. Вычисление вектора света и расстояния

$$\vec{s} = \vec{P}_T - \vec{P}_L$$

Вектор света $\vec{s}$ показывает направление от рассматриваемой точки P_T к источнику света P_L . Расстояние это просто длина этого вектора.

$$R^2 = \|\vec{s}\|^2$$

4. Вычисление углов альфа и тета

$$\cos \alpha = \frac{\vec{s} \cdot \vec{N}}{\|\vec{s}\|}, \cos \theta = \frac{\vec{s} \cdot \vec{O}}{\|\vec{s}\|}$$

Косинус угла между двумя векторами можно найти с помощью их скалярного произведения. $\cos \alpha$ - это косинус угла между вектором света s и вектором нормали N . Он показывает, насколько "прямо" свет падает на поверхность.

$\cos \theta$ - это косинус угла между вектором света s и осью источника света O . Этот угол нужен для вычисления диаграммы излучения источника.

5. Вычисление «цветной» интенсивности источника

$$I(RGB, \vec{s}) = I_0(RGB) \cos \theta$$

6. Конечное вычисление освещенности

$$E(RGB, \vec{P}_T) = \frac{I(RGB, \vec{s}) \cos \alpha}{R^2}$$

R^2 - квадрат расстояния от источника до точки. Это показывает, как освещённость ослабевает по мере удаления от источника.

Программа для расчета освещенности

```
#include <math.h>
#include <stdio.h>

typedef struct {
    float x;
    float y;
    float z;
} Vector3;

typedef struct {
    float r;
    float g;
    float b;
} Color;

// Вычитание векторов
Vector3 subtract(const Vector3 a, const Vector3 b) {
    Vector3 result;
    result.x = a.x - b.x;
    result.y = a.y - b.y;
    result.z = a.z - b.z;
    return result;
}
```

```

// Сложение векторов
Vector3 add(const Vector3 a, const Vector3 b) {
    Vector3 result;
    result.x = a.x + b.x;
    result.y = a.y + b.y;
    result.z = a.z + b.z;
    return result;
}

// Умножение вектора на скаляр
Vector3 multiply_scalar(const Vector3 v, const float scalar) {
    Vector3 result;
    result.x = v.x * scalar;
    result.y = v.y * scalar;
    result.z = v.z * scalar;
    return result;
}

// Скалярное произведение
float dot(const Vector3 a, const Vector3 b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

// Векторное произведение
Vector3 cross(const Vector3 a, const Vector3 b) {
    Vector3 result;
    result.x = a.y * b.z - a.z * b.y;
    result.y = a.z * b.x - a.x * b.z;
    result.z = a.x * b.y - a.y * b.x;
    return result;
}

// Длина вектора
float magnitude(const Vector3 v) {
    return sqrtf(dot(v, v));
}

// Нормализация вектора
Vector3 normalize(const Vector3 v) {
    const float mag = magnitude(v);
    if (mag == 0.0f) {
        const Vector3 zero = {0.0f, 0.0f, 0.0f};
        return zero;
    }
    return multiply_scalar(v, 1.0f / mag);
}

// Перевод в глобальные координаты
Vector3 compute_PT(const Vector3 P0, const Vector3 P1, const Vector3 P2,
const float x, const float y) {
    const Vector3 dir1 = subtract(P1, P0);
    const Vector3 dir2 = subtract(P2, P0);
    const Vector3 offset = add(multiply_scalar(dir1, x),
multiply_scalar(dir2, y));
    return add(P0, offset);
}

// Вычисление нормали к плоскости треугольника
Vector3 compute_N(const Vector3 P0, const Vector3 P1, const Vector3 P2) {
    const Vector3 v1 = subtract(P1, P0);

```

```

    const Vector3 v2 = subtract(P2, P0);
    const Vector3 cross_prod = cross(v1, v2);
    return normalize(cross_prod);
}

// Вычисление вектора s
Vector3 compute_s(const Vector3 PT, const Vector3 PL) {
    return subtract(PL, PT);
}

// Вычисление cos theta
float compute_cos_theta(const Vector3 s, const Vector3 O) {
    const Vector3 norm_O = normalize(O);
    const float mag_s = magnitude(s);
    if (mag_s == 0.0f) return 0.0f;
    return fmaxf(0.0f, dot(multiply_scalar(normalize(s), -1.0f),
norm_O));
}

// Вычисление cos alpha
float compute_cos_alpha(const Vector3 s, const Vector3 N) {
    const float mag_s = magnitude(s);
    if (mag_s == 0.0f) return 0.0f;
    return fmaxf(0.0f, dot(normalize(s), N));
}

// Вычисление интенсивности
Color compute_I(const Color IO, const float cos_theta) {
    Color result;
    result.r = IO.r * cos_theta;
    result.g = IO.g * cos_theta;
    result.b = IO.b * cos_theta;
    return result;
}

// Вычисление освещенности
Color compute_E(const Color I, const float cos_alpha, const float
R_squared) {
    if (R_squared <= 0.0f) return I;
    const float factor = cos_alpha / R_squared;
    Color result;
    result.r = I.r * factor;
    result.g = I.g * factor;
    result.b = I.b * factor;
    return result;
}

// Вычисление освещённости в точке
Color compute_illumination(const Vector3 P0, const Vector3 P1, const
Vector3 P2,
                        const float x, const float y,
                        const Vector3 PL, const Vector3 O, const Color
I0) {
    const Vector3 PT = compute_PT(P0, P1, P2, x, y);
    const Vector3 N = compute_N(P0, P1, P2);
    const Vector3 s = compute_s(PT, PL);
    const float R = magnitude(s);
    const float R_squared = R * R;

    const float cos_theta = compute_cos_theta(s, O);

```

```

    const float cos_alpha = compute_cos_alpha(s, N);

    const Color I = compute_I(I0, cos_theta);
    return compute_E(I, cos_alpha, R_squared);
}

int main(void) {
    // Вершины треугольника
    const Vector3 P0 = {0.0f, 0.0f, 0.0f};
    const Vector3 P1 = {4.0f, 0.0f, 0.0f};
    const Vector3 P2 = {2.0f, 4.0f, 2.0f};

    // Позиция источника света над треугольником
    const Vector3 PL = {-2.0f, 1.0f, 3.0f};

    // Начальная интенсивность света
    const Color I0 = {50.0f, 100.0f, 80.0f};

    // Направление оси света
    const Vector3 O = {0.0f, 0.0f, -1.0f};

    for (int y = 0; y <= 4; y++) {
        for (int x = 0; x <= 4; x++) {
            const Color E = compute_illumination(P0, P1, P2, (float)x/4,
            (float)y/4, PL, O, I0);
            printf("(%.2f, %.2f, %.2f) ", E.r, E.g, E.b);
        }
        printf("\n");
    }

    return 0;
}

```

Пример входных данных и результаты

1. $I_0(RGB) = (50, 100, 80)$

2. $\vec{O} = (0, 0, -1)$

3. $\vec{P}_L = (-2, 1, 3)$

4. $\vec{P}_0 = (0, 0, 0)$

5. $\vec{P}_1 = (4, 0, 0)$

6. $\vec{P}_2 = (2, 4, 2)$

7. $x_1 = 0.0, y_1 = 0.0$

$x_2 = 0.25, y_2 = 0.25$

$x_3 = 0.50, y_3 = 0.50$

$x_4 = 0.75, y_4 = 0.75$

$x_5 = 1.0, y_5 = 1.0$

Вычисленные значения освещенности $E(RGB, \vec{P}_T)$ для точек, заданных локальными координатами:

$x \backslash y$	0.0	0.25	0.50	0.75	1.0
0.0	(1.71, 3.42, 2.74)	(0.93, 1.86, 1.49)	(0.50, 0.99, 0.79)	(0.27, 0.55, 0.44)	(0.16, 0.32, 0.25)
0.25	(1.79, 3.58, 2.86)	(0.82, 1.63, 1.31)	(0.40, 0.80, 0.64)	(0.21, 0.42, 0.34)	(0.12, 0.24, 0.19)
0.50	(1.14, 2.28, 1.83)	(0.51, 1.01, 0.81)	(0.25, 0.50, 0.40)	(0.13, 0.27, 0.21)	(0.08, 0.15, 0.12)
0.75	(0.49, 0.98, 0.78)	(0.24, 0.48, 0.38)	(0.13, 0.25, 0.20)	(0.07, 0.14, 0.11)	(0.04, 0.09, 0.07)

1.0	(0.17, 0.33, 0.26)	(0.09, 0.18, 0.15)	(0.05, 0.11, 0.08)	(0.03, 0.06, 0.05)	(0.02, 0.04, 0.03)
-----	--------------------	--------------------	--------------------	--------------------	--------------------

Вывод

В результате выполнения работы был изучен и реализован алгоритм вычисления освещенности точки на плоскости от источника света. Рассмотрены основные этапы вычислений, включая преобразование координат, определение нормали к плоскости, вычисление вектора до источника света и углов между векторами.